Java Numeric Types

Storing Integers and Floating-Point Decimals Correctly

1. The Two Number Families

Integers

Whole numbers without any decimal points. Used for counting, years, and IDs.

- byte, short,
- int, long

Floating-Point

Numbers containing decimals. Used for prices, weights, and high-precision math.

- float, double

2. Integer Type Reference

Type	Size	Range	Notes
byte	1 Byte	-128 to 127	Very small memory footprint.
short	2 Bytes	~32k to 32k	Small numbers.
int	4 Bytes	~2 Billion	**Default for whole numbers.**
long	8 Bytes	Vast	Needs `L` at the end.

💡 **Pro Tip:** Use `int` for almost everything unless you have a specific reason to save memory (arrays) or need cosmic-scale numbers (long).

3. Floating-Point Decimals

Type	Size	Precision	Syntax
float	4 Bytes	6-7 Digits	`5.75f` (Needs `f`)
double	8 Bytes	15-16 Digits	`19.99` (Default decimal)

4. Scientific Notation

Java can handle very large or small numbers using the exponential e (meaning "times 10 to the power of").

```
float f1 = 35e3f; // 35,000
double d1 = 1.2E4; // 12,000
```

5. Implementation Example

```
public class Main {
  public static void main(String[] args) {
    int age = 25;
    long pop = 8000000000L; // Big number, notice the L
    float price = 19.99f; // Decimal, notice the f
    double pi = 3.1415926;

    System.out.println("Price: " + price);
  }
}
```

SUMMARY CHECKLIST

1. Integers = Whole #s | Floating-point = Decimals.
2. Default Integer: `int` | Default Decimal: `double`.
3. Don't forget the **'L'** for long or the **'f'** for float!
4. Floating-point types (float/double) can store whole numbers, but Integer types (int) **cannot** store decimals.